AI INFRASTRUCTURE

kagent Baseline Test

2026-11-10 Author: Ciaran Donegan kagent v0.10.6 Status: Continuous

---

Fleet footprint baseline for the kagent agent framework on a 5-node bare-metal cluster. 102 pods deployed concurrently across 6 agent types, all running simultaneously. Idle memory and CPU measured from Prometheus. Establishes the per-pod resource floor for capacity planning a 100-agent fleet on EKS.

All Go agent types converge at 6.6-7.7 MB idle RSS regardless of system prompt size, tools, or conversation state. Python ADK pods idle at 219 MB -- 33x more. All 6 types reached full concurrent deployment; linear scaling confirmed across the board.

Test Environment

Local 5-node bare-metal Kubernetes cluster: 2 schedulable worker nodes (wk1: 4 CPU, 10.5 GB; wk4: 12 CPU, 38.8 GB) and 3 control plane nodes with untolerated taints. kagent installed via Helm from the upstream OCI chart. Model gateway: LiteLLM (proxying Claude Sonnet 5.5 via the Anthropic API).

All 6 agent deployments scaled to 17 replicas simultaneously (102 target pods total) in namespace `kagent`. All 102 pods reached Ready. Prometheus memory and CPU scraped after a 13-minute settle window with all pods running concurrently at idle.

Key Figures

102-POD FLEET MEMORY

4.2 GB

Mixed fleet idle -- 6 agent types, all running concurrently

PYTHON VS GO MEMORY

33x

219 MB vs 6.6 MB per pod at idle

GO RUNTIME IDLE

~7 MB

All 5 Go types converge -- system prompt size and tools have no effect on idle memory

100 PYTHON-MINIMAL PODS

21.4 GB

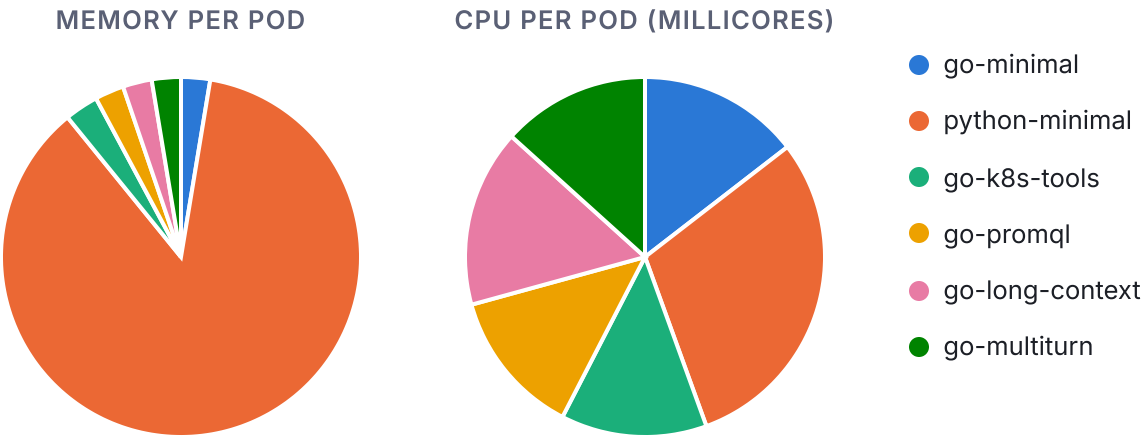
vs 660 MB for 100 go-minimal pods

Fleet Footprint

All 6 agent types scaled to 17 replicas concurrently (102 pods). All 102 reached Ready. Prometheus memory and CPU scraped at idle after a 13-minute settle window.

AGENT TYPE	RUNTIME	MEMORY / POD	CPU / POD (M)	100-POD MEMORY	100-POD CPU (M)
go-minimal	Go	6.6 MB	0.77	660 MB	77
python-minimal	Python	218.8 MB	1.59	21.4 GB	159
go-k8s-tools	Go	7.7 MB	0.70	766 MB	70
go-promql	Go	6.6 MB	0.70	659 MB	70
go-long-context	Go	6.6 MB	0.85	660 MB	85
go-multiturn	Go	6.6 MB	0.71	660 MB	71

Idle-only: pods were not serving requests during measurement. Memory figures are RSS. CPU figures are Prometheus rate over a 2-minute window. Full concurrent deployment confirmed across all 6 types.



Findings

1. Go vs Python runtime memory

All 5 Go-runtime agent types idle at 6.6-7.7 MB RSS regardless of system prompt size, tool definitions, or conversation state. Python-runtime pods idle at 219 MB -- 33x more than go-minimal. The difference is the Python interpreter and google-adk package overhead, present in every pod regardless of what the agent does. At 100 pods, Go costs 659-766 MB; Python costs 21.4 GB. The kagent controller itself is always Go; the runtime choice only affects each agent pod.

2. Tool schema cost is in tokens, not footprint

go-k8s-tools carries full Kubernetes tool schemas in its system prompt yet idles at 7.7 MB -- near-identical to go-minimal at 6.6 MB. Tool definitions, system prompt size, and conversation state have no effect on idle pod footprint. For capacity planning, all Go types can be treated as equivalent at idle regardless of tool configuration.

3. Fleet is memory-bound at idle

CPU usage across all 6 types is negligible at idle -- 0.70-1.59 m per pod. The fleet sizing constraint is entirely memory. A mixed 100-agent fleet (all 6 types) requires approximately 25 GB RAM. A Go-only fleet of 100 agents requires under 800 MB. Runtime choice is the dominant capacity planning variable.

Gaps vs Production Stack

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
llm-d inference layer (LLMInferenceService)	Production uses KServe LLMInferenceService v0.20.0 (a separate controller from classic KServe) with the llm-d endpoint picker (`llm-d-router-endpoint-picker v0.9.0`). The picker does cache-aware routing across GPU pods and disaggregated prefill/decode. Classic KServe is not used in production for LLM serving.	Test used LiteLLM proxying Ollama. Classic KServe is running on the local cluster (Ollama InferenceService in ai-platform), but the LLMInferenceService CRD and llm-d endpoint picker are not installed. No cache-aware routing or disaggregated serving in this test.	Partial - classic KServe already present. Installing llmops-kserve (the LLMInferenceService controller + llm-d) requires the additional chart from the repo. Medium effort, low disruption risk since it adds new CRDs alongside existing ones.
Envoy Gateway + Agent Router	Single front door, EKS token auth, routing to `/api/a2a/kagent/<name>/` . The AI Gateway ext_proc sidecar counts tokens per request and writes to Valkey quota counters.	Agents called via local Cilium gateway. No auth, no quota enforcement, no token counting at the gateway layer.	No - llmops-gateway chart needs Terraform-provisioned AWS resources (Transit Gateway, VPC endpoints, EKS Pod Identity).
OPA	Two enforcement points: (1) agent access - checks caller's tenant registry for the agent name, 403 otherwise; (2) model access - checks agent's model calls against the tenant's allowed models list. Runs as an Envoy ext_authz filter, reloads registry within ~1 minute of a change.	No enforcement in this test. Any caller can reach any agent and any model. Token usage is not charged to a tenant quota.	Partial - OPA can be deployed locally with the tenant registry, but without the Envoy gateway routing through it, it won't be in the request path.
Valkey (ElastiCache) quota counters	Every response is charged to the tenant's token quota in Valkey. The AI Gateway ext_proc writes the token cost; the quota check blocks requests over-limit before they reach the model.	No quota enforcement. No per-tenant usage accounting. Test ran uncapped against a shared LiteLLM instance.	Partial - Valkey (Redis-compatible) can run locally. Without the AI Gateway ext_proc writing to it, quota enforcement is not in the request path.

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
vClusters **(bridged)**	One virtual cluster per tenant, running as pods in the host namespace. Gives each tenant an isolated Kubernetes API server - they cannot see other tenants or host cluster internals. Agents live on the host cluster; tenants call them from inside their vCluster via the gateway.	Bridged - vCluster v0.37.6 installed, tenant vCluster `cappuccino` running in `vcluster-cappuccino`. Called `baseline-go-minimal` from inside: state completed, 639/115 tokens. Namespace isolation confirmed - only default/kube-system visible from inside. Remaining gap: EKS OIDC JWT auth and the vCluster token injection (MutatingAdmissionPolicy) are not configured.	Done - `vcluster create cappuccino --namespace vcluster-cappuccino`. Network connectivity to host cluster gateway works without additional config.
AWS Bedrock - Claude Sonnet 5 **(bridged)**	Production model is `us.anthropic.claude-sonnet-5` via Bedrock regional inference profile. Reliable tool-call schema compliance (no ask_user misfires). Latency is lower than local Ollama.	Bridged with `claude-sonnet-5-5` via the Anthropic API through LiteLLM. Zero tool-call misfires. Token counts and cost estimates in this test use real Sonnet 5.5 rates ($2.00/$10.00 per MTok). Not identical to a managed Bedrock inference profile, but model behaviour and cost profile are equivalent for this test.	Done - LiteLLM ModelConfig updated to `claude-sonnet-5-5`. ANTHROPIC_API_KEY injected via secret. Remaining gap: Bedrock IAM + regional inference profile routing not tested.
llmops-agents chart	Hardened kagent: external Postgres for agent state, namespaced RBAC, apiKeyPassthrough (caller's EKS token forwarded to the model gateway).	Vanilla upstream kagent chart used. No external database, no caller token passthrough. Agent state lives in the controller's in-cluster store.	Yes - chart is in the repo. llmops-agents wraps upstream kagent with an external Postgres database (already running via kagent-postgresql). Medium effort to swap in. ~1-2 hours.
Karpenter + GPU nodes (EC2 G6e / L40S)	GPU node autoscaling for vLLM. DCGM exporter provides GPU metrics via NVIDIA GPU Operator v26.7.0.	Not relevant for kagent agent pods (CPU-only). Not relevant for kagent agent pod footprint. GPU node performance affects inference latency, not idle pod resource usage.	N/A - requires EKS and AWS GPU instances.

These results establish the idle pod footprint for kagent agent pods -- memory and CPU per type at scale. Footprint figures should hold on EKS since kagent is model-agnostic and the Go runtime overhead is independent of the inference layer. The gaps above affect request-time cost and latency, not idle footprint.

Production Gateway Architecture

The Envoy Gateway enforces three things in sequence for every request:

1. Identity

The caller presents a Kubernetes ServiceAccount token. The gateway verifies it and strips any identity headers the client tries to inject. Tenants in vClusters get the token automatically.

2. Authorisation

OPA checks whether this tenant is allowed to call this agent or model. Tenant permissions are managed in a config file and update live without restarting the gateway.

3. Quota

Token usage is counted per response and charged against the tenant's quota in a shared counter store. Requests from tenants over their limit are blocked before reaching the model.